

คู่มือสร้างไเกอร์เคาน์เตอร์ด้วย Arduino Pro Mini และ LND712

สวัสดีครับ! บทความนี้จะพาคุณไปสร้าง ไเกอร์เคาน์เตอร์ แบบพกพา วัดรังสีอัลฟา เบตา แกมมา และรังสีเอกซ์ ได้ โดยใช้หลอด LND712 ร่วมกับ Arduino Pro Mini 5V พร้อมฟังก์ชันครบครัน ไม่ว่าจะเป็นจอแสดงผล OLED แบบ I2C, การปรับแก้ค่าภูมิหลัง (background correction), การแปลงหน่วย (conversion factor), การตั้งค่าแจ้งเตือนด้วยเสียงเมื่อเจอระดับรังสีสูงผิดปกติ, การควบคุมด้วยปุ่มกด 3 ปุ่ม และการส่งข้อมูลผ่าน Bluetooth ไปยังสมาร์ทโฟน บทความนี้จะอธิบายทั้งฮาร์ดแวร์ หลักการทำงาน และได้ตัวอย่างละเอียดเป็นภาษาไทย

๑ ส่วนประกอบฮาร์ดแวร์

รายการอุปกรณ์ที่ต้องใช้ มีดังนี้

1. Arduino Pro Mini 5V (16MHz) – บอร์ดไมโครคอนโทรลเลอร์หลัก
2. หลอดไเกอร์-มิลเลอร์ LND712 – ตรวจจับรังสีอัลฟา เบตา แกมมา และรังสีเอกซ์
3. โมดูลจ่ายแรงดันสูง 400-550V – จ่ายไฟให้หลอดทำงาน (ควรเป็นโมดูลที่มีมาตรฐาน TTL pulse output)
4. จอ OLED ขนาด 0.96 นิ้ว รุ่น SSD1306 แบบ I2C – แสดงผล CPM, $\mu\text{Sv/h}$ และสถานะการตั้งค่า
5. โมดูล Bluetooth HC-05 หรือ HC-06 – สำหรับส่งข้อมูลไปยังสมาร์ทโฟน
6. ปุ่มกดแบบ Tactile Switch 3 ตัว – ใช้สำหรับเมนูตั้งค่า: Select, Up, Down
7. ลำโพงเสียงหรือ Buzzer แบบ Passive – ส่งเสียงแจ้งเตือนเมื่อรังสีเกินค่าที่กำหนด
8. แหล่งจ่ายไฟ – แบตเตอรี่ Li-ion 3.7V พร้อมโมดูลชาร์จ USB (Power Bank ก็ได้)
9. สายไฟ อุปกรณ์บัดกรี และแผ่นวงจรเบรตบอร์ดสำหรับทดสอบ

⚠ ข้อควรระวังด้านความปลอดภัย

โมดูลจ่ายแรงดันสูงสร้างแรงดันไฟฟ้าสูงถึง 400-550V ซึ่งอาจเป็นอันตรายได้ ควรใช้ความระมัดระวังขณะต่อวงจร และตรวจสอบฉนวนหุ้มทุกจุดให้เรียบร้อย

๒ การเชื่อมต่อฮาร์ดแวร์

กำหนดหมายเลขขาเชื่อมต่อดังนี้

อุปกรณ์	ขาสัญญาณ	เชื่อมต่อขา Arduino
โมดูลจ่ายแรงดันสูง	* Pulse Output	D2 (Interrupt 0)
OLED I2C	SDA	A4
OLED I2C	SCL	A5
ปุ่ม Select	Output	D3
ปุ่ม Up	Output	D4
ปุ่ม Down	Output	D5
Buzzer	Signal	D6
Bluetooth HC-05	TX → Arduino	D7 (Software Serial)
Bluetooth HC-05	RX ← Arduino	D8 (Software Serial)

หมายเหตุ: OLED ใช้สาย I2C แบบ 4 เส้น: VCC, GND, SDA, SCL หากจอของคุณใช้แรงดัน 3.3V ควรใช้ Level Shifter สำหรับขา SDA/SCL (Arduino Pro Mini ใช้ logic 5V)

แผนภาพการเชื่อมต่ออย่างง่ายสามารถเขียนเพิ่มเติมในส่วนอ้างอิงได้

หลักการทำงานและการคำนวณ

1. การนับพัลส์ (Pulse Counting)

หลอด LND712 จะสร้างพัลส์แรงดันสั้น ๆ ทุกครั้งที่อนุภาครังสีผ่านเข้าไป โมดูลจ่ายแรงดันสูงจะปรับพัลส์นี้ให้อยู่ในรูปแบบสัญญาณ TTL (0-5V) แล้วส่งไปยังขา D2 ของ Arduino ซึ่งเราจะใช้ **interrupt** เพื่อตรวจจับและนับจำนวนพัลส์อย่างแม่นยำ

2. การคำนวณ CPM

CPM (Counts Per Minute) คือจำนวนพัลส์ที่เกิดขึ้นใน 1 นาที ในโค้ดเราจะเก็บค่าพัลส์ทุกวินาที (Counts Per Second, CPS) แล้วนำไปรวมในช่วง 60 วินาที หรือสามารถใช้วิธีคูณตัวอย่าง 10 วินาทีแล้วคูณด้วย 6 เพื่อให้แสดงผลรวดเร็วขึ้น

text

$$\text{CPM} = (\text{จำนวนพัลส์ที่ได้ใน } T \text{ วินาที}) \times (60 / T)$$

3. Conversion Factor (CPM $\rightarrow$ $\mu\text{Sv/h}$)

หลอดไกเกอร์แต่ละรุ่นมี Conversion Factor ที่แตกต่างกัน สำหรับ LND712 มีค่าที่นิยมใช้หลายค่า:

- **0.00833** – ใช้ในโปรเจกต์ Geiger Counter บน ESP32 ของ danjulio
- **0.00926** (108 CPM ต่อ 1 $\mu\text{Sv/h}$) – พบในเอกสารอ้างอิงบางแหล่ง
- **0.00233** – ตามตาราง Conversion Factors บางชุด

ในบทความนี้จะใช้ค่า 0.00833 เป็นค่าเริ่มต้น ซึ่งสามารถปรับเปลี่ยนได้ผ่านปุ่มตั้งค่า

text

$$\text{Dose } (\mu\text{Sv/h}) = \text{CPM} \times \text{Conversion_Factor}$$

4. Background Correction

ค่ารังสีพื้นหลัง (Background Radiation) มีค่าประมาณ **15–30 CPM** ขึ้นอยู่กับสถานที่ เพื่อให้ค่าที่วัดได้แม่นยำยิ่งขึ้น โค้ดจะทำการลบค่าพื้นหลังโดยอัตโนมัติ โดยเก็บค่าเฉลี่ยเมื่อไม่พบแหล่งรังสีเพิ่มเติม

text

$$\text{Adjusted_CPM} = \text{CPM} - \text{Background_CPM}$$

หาก Adjusted_CPM ติดลบ ให้แสดงผลเป็น 0 CPM / 0.00 $\mu\text{Sv/h}$

5. การแจ้งเตือนระดับรังสี

เมื่อ Dose rate สูงเกินค่าที่ตั้งไว้ (Alert Threshold) Arduino จะส่งสัญญาณไปที่ Buzzer เพื่อส่งเสียงเตือน โดยสามารถกำหนดความถี่และระยะเวลาการติดต่อกันได้

📄 โค้ดสมบูรณ์พร้อมคำอธิบาย

ขั้นตอนที่ 1: ติดตั้งไลบรารีที่จำเป็น

ก่อนอัปโหลดโค้ด ต้องติดตั้งไลบรารีต่อไปนี้ใน Arduino IDE:

- **Adafruit SSD1306** – สำหรับจอ OLED

- Adafruit GFX – ฟังก์ชันกราฟิกสำหรับจอ
- SoftwareSerial – สำหรับติดต่อ Bluetooth (มีใน Arduino IDE แล้ว)

ขั้นตอนที่ 2: โค้ดหลัก

cpp

/*

⌘ ไกเกอร์เคาน์เตอร์ด้วย Arduino Pro Mini + LND712

ฟีเจอร์:

- นับพัลส์จากหลอด LND712 (ขา D2, interrupt)
- แสดง CPM และ $\mu\text{Sv/h}$ บนจอ OLED I2C
- ปุ่มปรับค่า Conversion Factor และ Alert Threshold
- ส่งข้อมูลทาง Bluetooth ไปยังสมาร์ทโฟน
- สัญญาณเสียงเตือนเมื่อรังสีเกินค่า Alert Threshold

*/

#include <Wire.h>

#include <Adafruit_SSD1306.h>

#include <Adafruit_GFX.h>

#include <SoftwareSerial.h>

// ----- ตั้งค่าหน้าจอ OLED -----

#define SCREEN_WIDTH 128

#define SCREEN_HEIGHT 64

#define OLED_ADDR 0x3C // I2C address ของจอ SSD1306

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ----- กำหนดขาต่ออุปกรณ์ -----

#define GEIGER_PIN 2 // ขาสัญญาณ Pulse จากโมดูล HV (interrupt 0)

```

#define BUTTON_SEL 3 // ปุ่ม Select (Enter / OK)
#define BUTTON_UP 4 // ปุ่ม เพิ่มค่า
#define BUTTON_DN 5 // ปุ่ม ลดค่า
#define BUZZER_PIN 6 // ขาควบคุมเสียงเตือน
#define BT_RX 7 // ขา RX ของ HC-05 (ต่อกับ TX ของ HC-05)
#define BT_TX 8 // ขา TX ของ HC-05 (ต่อกับ RX ของ HC-05)

```

```

SoftwareSerial bluetooth(BT_RX, BT_TX); // RX, TX

```

```

// ----- ค่าคงที่หลัก -----

```

```

const unsigned long SAMPLE_TIME = 10000; // เก็บตัวอย่างทุก 10 วินาที (ms)
const float MULTIPLIER = 60000.0 / SAMPLE_TIME; // เปลี่ยน CPS เป็น CPM
const float DEFAULT_CONV_FACTOR = 0.00833; // Conversion factor สำหรับ LND712
const float DEFAULT_BACKGROUND = 20.0; // ค่ารังสีพื้นหลังเริ่มต้น (CPM)
const float DEFAULT_ALERT_THRESHOLD = 1.0; // ค่าเตือนเริ่มต้น (µSv/h)

```

```

// ----- ตัวแปรส่วนกลาง -----

```

```

volatile unsigned long pulseCount = 0; // จำนวนพัลส์ที่ตรวจจับได้
unsigned long lastSampleTime = 0;
float cpm = 0.0;
float doseRate = 0.0;

```

```

// ตัวแปรสำหรับการตั้งค่า

```

```

float convFactor = DEFAULT_CONV_FACTOR;
float background = DEFAULT_BACKGROUND;
float alertThreshold = DEFAULT_ALERT_THRESHOLD;

```

// สถานะปุ่มและเมนู

int menuMode = 0; // 0 = หน้าหลัก, 1 = แก้ไข Conversion Factor, 2 = แก้ไข Background, 3 = แก้ไข Alert Threshold

bool buttonPressed = false;

unsigned long lastButtonTime = 0;

const unsigned long debounceDelay = 200; // กั้นการกระดิ่งของปุ่ม (ms)

// ตัวแปรสำหรับการเตือน

bool alertActive = false;

unsigned long lastAlertTime = 0;

const unsigned long alertDuration = 500; // เสียงดัง 0.5 วินาที

// -----

// ฟังก์ชัน Interrupt: ทำงานทุกครั้งที่เกิดพัลส์จากหลอดไทดัล

// -----

void countPulse() {

 pulseCount++;

}

// -----

// อ่านสถานะปุ่ม (พร้อม debounce)

// -----

int readButton(int pin) {

 if (digitalRead(pin) == LOW) { // สมมติปุ่มต่อแบบ Active Low (กด = LOW)

 if ((millis() - lastButtonTime) > debounceDelay) {

 lastButtonTime = millis();

 return 1;

 }

 }

```
return 0;
```

```
}
```

```
//-----
```

```
// แสดงข้อความแจ้งเตือนเมื่อรังสีเกินค่า Alert Threshold
```

```
//-----
```

```
void triggerAlert() {
```

```
    if (!alertActive) {
```

```
        alertActive = true;
```

```
        lastAlertTime = millis();
```

```
        tone(BUZZER_PIN, 2000, alertDuration); // เสียงความถี่ 2000Hz นาน 0.5 วินาที
```

```
    }
```

```
}
```

```
//-----
```

```
// อัปเดตหน้าจอ OLED แสดงค่า CPM,  $\mu\text{Sv/h}$  และสถานะ
```

```
//-----
```

```
void updateDisplay() {
```

```
    display.clearDisplay();
```

```
    display.setTextSize(1);
```

```
    display.setTextColor(SSD1306_WHITE);
```

```
    display.setCursor(0, 0);
```

```
// แสดง CPM และ Dose Rate
```

```
display.print("CPM: ");
```

```
display.println(cpm, 0);
```

```
display.print("Dose: ");
```

```
display.print(doseRate, 2);
```

```
display.println("  $\mu\text{Sv/h}$ ");
```

```
// แสดงค่า Alert Threshold และ Conversion Factor
```

```
display.print("Alert: ");
```

```
display.print(alertThreshold, 2);
```

```
display.print(" uSv/h");
```

```
display.setCursor(0, 40);
```

```
display.print("Conv: ");
```

```
display.print(convFactor, 5);
```

```
// หากกำลังอยู่ในโหมดเมนู ให้แสดงข้อความแจ้ง
```

```
if (menuMode != 0) {
```

```
    display.setCursor(0, 52);
```

```
    display.print("Menu Mode: ");
```

```
    switch(menuMode) {
```

```
        case 1: display.print("Conv Factor"); break;
```

```
        case 2: display.print("Background"); break;
```

```
        case 3: display.print("Alert Thresh"); break;
```

```
    }
```

```
}
```

```
display.display();
```

```
}
```

```
// -----
```

```
// ส่งข้อมูลทาง Bluetooth ไปยังสมาร์ทโฟน
```

```
// รูปแบบ: cpm=123.45;uSv/h=1.03
```

```
// -----
```

```
void sendBluetoothData() {
```

```
    char buffer[50];
```

```
    snprintf(buffer, sizeof(buffer), "cpm=%.0f;uSv/h=%.2f\n", cpm, doseRate);
```

```
    bluetooth.print(buffer);
```

```
}
```

```
//-----
```

```
// จัดการเมนูการตั้งค่าผ่านปุ่มกด
```

```
//-----
```

```
void handleMenu() {
```

```
  if (readButton(BUTTON_SEL)) {
```

```
    // กดปุ่ม Select: เปลี่ยนโหมดเมนู (วนกลับ)
```

```
    menuMode = (menuMode + 1) % 4;
```

```
    delay(200); // หน่วงเล็กน้อยเพื่อป้องกันการกดซ้ำ
```

```
  }
```

```
// ถ้าอยู่ในโหมดเมนู (menuMode != 0) ให้ปรับค่าด้วยปุ่ม Up/Down
```

```
  if (menuMode == 1) {
```

```
    if (readButton(BUTTON_UP)) convFactor += 0.0001;
```

```
    if (readButton(BUTTON_DN) && convFactor > 0) convFactor -= 0.0001;
```

```
  }
```

```
  else if (menuMode == 2) {
```

```
    if (readButton(BUTTON_UP)) background += 1.0;
```

```
    if (readButton(BUTTON_DN) && background > 0) background -= 1.0;
```

```
  }
```

```
  else if (menuMode == 3) {
```

```
    if (readButton(BUTTON_UP)) alertThreshold += 0.1;
```

```
    if (readButton(BUTTON_DN) && alertThreshold > 0) alertThreshold -= 0.1;
```

```
  }
```

```
}
```

```
//-----
```

```
// ฟังก์ชัน setup() ทำงานครั้งแรกเมื่อเริ่มต้น
```

```
//-----
```

```

void setup() {
  Serial.begin(9600); // Serial Monitor สำหรับดีบั๊ก
  bluetooth.begin(9600); // เริ่มการทำงานของBluetooth (HC-05 ปกติใช้ 9600 baud)

  // เริ่มต้นจอ OLED
  if(!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;); // หยุดทำงานหากจอแสดงผลไม่เริ่ม
  }
  display.clearDisplay();
  display.display();
  delay(1000);

  // ตั้งค่าขา
  pinMode(GEIGER_PIN, INPUT);
  pinMode(BUTTON_SEL, INPUT_PULLUP);
  pinMode(BUTTON_UP, INPUT_PULLUP);
  pinMode(BUTTON_DN, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);

  // เปิดใช้งาน Interrupt สำหรับขา D2 (ขา 2 = interrupt 0)
  attachInterrupt(digitalPinToInterrupt(GEIGER_PIN), countPulse, RISING);

  // แสดงข้อความเริ่มต้น
  display.clearDisplay();
  display.setCursor(0, 0);
  display.println("Geiger Counter");
  display.println("LND712 Ready");
  display.display();
}

```

```
delay(2000);
```

```
lastSampleTime = millis();
```

```
}
```

```
//-----
```

```
// ฟังก์ชัน loop() ทำงานวนซ้ำตลอดเวลา
```

```
//-----
```

```
void loop() {
```

```
    // จัดการเมนูและปุ่มกด
```

```
    handleMenu();
```

```
    // ทุก ๆ SAMPLE_TIME (10 วินาที) ให้คำนวณ CPM, Dose Rate และแสดงผล
```

```
    if (millis() - lastSampleTime >= SAMPLE_TIME) {
```

```
        // ปิด Interrupt ชั่วคราวขณะอ่านค่า pulseCount
```

```
        noInterrupts();
```

```
        unsigned long counts = pulseCount;
```

```
        pulseCount = 0;
```

```
        interrupts();
```

```
        // คำนวณ CPM และ Dose Rate
```

```
        cpm = counts * MULTIPLIER;
```

```
        float adjustedCPM = cpm - background;
```

```
        if (adjustedCPM < 0) adjustedCPM = 0;
```

```
        doseRate = adjustedCPM * convFactor;
```

```
        // ตรวจสอบการเตือน
```

```
        if (doseRate >= alertThreshold) {
```

```
            triggerAlert();
```

```

} else {
  alertActive = false;
}

```

```

// อัปเดตหน้าจอ OLED
updateDisplay();

```

```

// ส่งข้อมูลทาง Bluetooth
sendBluetoothData();

```

```

// รีเซ็ตเวลา
lastSampleTime = millis();
}

```

```

// วนลูปเล็กน้อยเพื่อให้ปุ่มทำงานได้ทันที
delay(50);
}

```

คำอธิบายโค้ดสำคัญ

ส่วนของโค้ด

คำอธิบาย

countPulse()

ฟังก์ชัน Interrupt ที่เพิ่มค่า pulseCount ทุกครั้งที่มีพัลส์จากหลอดไกเกอร์

handleMenu()

ตรวจสอบปุ่มกด Select เพื่อเปลี่ยนโหมดเมนู และปุ่ม Up/Down เพื่อปรับค่า Conversion Factor, Ba Alert Threshold

updateDisplay()

แสดงค่า CPM, Dose Rate ($\mu\text{Sv/h}$), ค่า Alert Threshold และ Conversion Factor บนจอ OLED

sendBluetoothData()

ส่งข้อมูลในรูปแบบ cpm=ค่าที่ได้; $\mu\text{Sv/h}$ =ค่าที่ได้ ผ่าน Bluetooth ไปยังสมาร์ทโฟน

ส่วนของโค้ด

คำอธิบาย

triggerAlert()

เปิดเสียง Buzzer เมื่อ Dose Rate สูงกว่า Alert Threshold

การปรับแต่ง Conversion Factor

- ค่าเริ่มต้นที่ใช้ในโปรเจกต์อ้างอิงคือ 0.00833 (จาก LND712 CPM $\rightarrow$ μ Sv/h)
- หากต้องการเปลี่ยน ให้กดปุ่ม Select เข้าสู่โหมด "Conv Factor" จากนั้นใช้ปุ่ม Up/Down ปรับค่า
- การปรับค่า Background ช่วยให้ค่าที่แสดงมีความแม่นยำมากขึ้นโดยลบรังสีพื้นหลังออก

📱 การรับข้อมูลบนสมาร์ทโฟนผ่าน Bluetooth

โค้ดจะส่งข้อมูลทุก 10 วินาทีในรูปแบบ:

text

cpm=124.00;uSv/h=1.03

คุณสามารถรับข้อมูลนี้ผ่านแอปพลิเคชันบนสมาร์ทโฟน เช่น:

- **Serial Bluetooth Terminal** (Android / iOS)
- **Arduino Bluetooth Data Logger** หรือแอปพลิเคชันที่คุณพัฒนาเอง

ขั้นตอนการใช้งาน:

1. จับคู่ Bluetooth กับโมดูล HC-05 (รหัสผ่านเริ่มต้นคือ 1234 หรือ 0000)
2. เปิดแอปพลิเคชัน Serial Terminal
3. เลือกเชื่อมต่อกับ HC-05
4. ข้อมูลจะแสดงเป็นข้อความทุก 10 วินาที

หมายเหตุ: หากใช้ HC-06 (Slave-only) ก็สามารถใช้ได้แต่ได้ เพียงแต่ไม่สามารถรับคำสั่งจากสมาร์ทโฟนมายัง Arduino ได้

✳ คำแนะนำที่เป็นประโยชน์

การทดสอบระบบ

1. ทดสอบทีละส่วน – ก่อนต่ออุปกรณ์ทั้งหมด ให้ทดสอบจอ OLED, ปุ่มกด, Buzzer และ Bluetooth แยกกันก่อน
2. ตรวจสอบสัญญาณจากหลอดไทเกอร์ – ใช้ oscilloscope หรือ multimeter ที่วัดความถี่ได้ เพื่อดูว่ามีพัลส์เกิดขึ้นจริง
3. ตรวจสอบแรงดันไฟฟ้า – หลอด LND712 ต้องการแรงดันในช่วง **450-650V** ควรตั้งค่าโมดูลจ่ายแรงดันสูงให้ได้ประมาณ 500V
4. หลีกเลียงสัญญาณรบกวน – เก็บสายสัญญาณ Pulse ให้สั้นที่สุด และใช้การกรองสัญญาณด้วยตัวต้านทาน pull-down หากจำเป็น

การปรับเทียบ (Calibration)

- ค่า Conversion Factor อาจแตกต่างกันไปตามแหล่งอ้างอิง ควรเทียบกับเครื่องมือมาตรฐานหากมีความจำเป็น
- หลอด LND712 มี Background ประมาณ **15-30 CPM** หากค่าที่ได้สูงกว่านี้มากในที่โล่งแจ้ง แสดงว่ามีสัญญาณรบกวนหรือการตั้งค่า Background ไม่ถูกต้อง
- ควรตั้งค่า Background โดยวัดในที่ที่ไม่มีแหล่งรังสีเพิ่มเติม (เช่น ห้องปิด) เป็นเวลา 10-15 นาที แล้วนำค่าเฉลี่ยมาใช้

การจัดการพลังงาน

- Arduino Pro Mini 5V กินกระแสประมาณ 20-30 mA + จอ OLED (~20 mA) + Bluetooth (~30 mA) รวมแล้วประมาณ 80-100 mA
- แบตเตอรี่ 18650 ขนาด 2000 mAh จะใช้งานได้ประมาณ 20 ชั่วโมง
- หากต้องการประหยัดพลังงาน ให้ปิด Bluetooth เมื่อไม่ใช้งาน หรือใช้โหมด sleep ของ Arduino

การแก้ไขปัญหาเบื้องต้น

ปัญหา	สาเหตุและแนวทางแก้ไข
จอ OLED ไม่แสดงผล	ตรวจสอบการเชื่อมต่อ I2C, ลองสแกนหา I2C address ด้วยโค้ด Scanner
Bluetooth ไม่เชื่อมต่อ	ตรวจสอบสาย TX/RX และ baud rate (9600)

ปัญหา

สาเหตุและแนวทางแก้ไข

ค่า CPM สูงผิดปกติ

ตรวจสอบสัญญาณรบกวน, ลองเพิ่มตัวกรอง (RC filter) ที่ขาสัญญาณ

เสียงเตือนไม่ดัง

ตรวจสอบขา Buzzer และ polarity (ถ้าใช้ active buzzer)

📖 อ้างอิงเพิ่มเติม

- LND712 Datasheet: [LND Inc.](#)
- DIY Geiger Counter Reference: [Google Sites](#)
- ตัวอย่างโค้ดบน GitHub: [danjulio/Geiger-Counter](#)
- Conversion Factors สำหรับหลอดไกเกอร์: [Apollo-NG](#)

📌 สรุป

คุณได้เรียนรู้วิธีการสร้างไกเกอร์เคาน์เตอร์ด้วย Arduino Pro Mini และหลอด LND712 ครบทุกฟังก์ชัน ตั้งแต่การนับพัลส์ การคำนวณ CPM และ Dose Rate การปรับแก้ค่าพื้นหลังและ Conversion Factor การตั้งค่าแจ้งเตือนด้วยเสียง การควบคุมเมนูด้วยปุ่มกด และการส่งข้อมูลผ่าน Bluetooth ไปยังสมาร์ตโฟน โค้ดที่ให้ไว้สามารถนำไปปรับใช้ได้ทันที และสามารถพัฒนาเพิ่มเติมได้ เช่น การบันทึกข้อมูลลง SD Card, การแสดงกราฟแนวโน้ม หรือการเพิ่มเซนเซอร์วัดอุณหภูมิและความชื้น

หากมีข้อสงสัยเพิ่มเติม สามารถสอบถามได้ในช่องแสดงความคิดเห็นด้านล่างครับ!